

Lab Assignment 1: INTRO TO VERILOG & RTL Coding Styles

Module: Verilog & Logic Synthesis

Instructor Note:

Complete the following assignment and lab practical. Your responses must demonstrate a clear understanding of how Verilog translates to actual hardware behavior, avoiding software-centric assumptions.

Part 1: The Philosophy of Hardware Description

1. Why We Use HDLs

Before the 1980s, engineers designed circuits by drawing them by hand.

- Why did drawing circuits become too difficult as technology advanced?
- How did using Hardware Description Languages (HDLs) fix this problem?

2. Hardware vs. Software

Verilog is not like standard programming languages such as Python or C++.

- What is the difference between an HDL and a software language?
- In your answer, explain how each language handles tasks happening at the same time (parallel) versus one after another (serial).

3. The Evolution of Verilog

The IEEE 1800-2009 standard was a big step forward for hardware design.

- Which two hardware languages were combined to make this standard?
- Why was it helpful for engineers to combine these two languages into one?

Part 2: Navigating the Abstraction Hierarchy

1. The Y-Chart Domains

The Gajski-Kuhn Y-Chart shows three different ways (or domains) to look at a digital system.

- List the names of these three domains.

2. Choosing the Right Level

Register-Transfer Level (RTL) is the standard abstraction level used for designing real hardware like FPGAs and chips.

- Why do engineers use RTL for creating hardware instead of the higher Behavioral level? (Hint: Think about what the synthesis tool needs to know).

3. Comparison of Verilog Abstraction Levels.

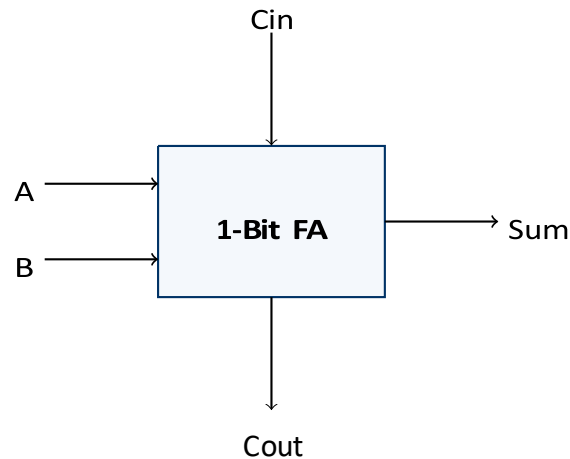
Verilog supports four primary abstraction levels to describe digital hardware: Behavioral (Algorithmic), Dataflow, Gate, and Structural.

- **Compare the Verilog abstraction levels** in terms of the **primary focus, used Verilog keywords and constructs, typical design suitability, and synthesizability** of each level

Part 3: Multi-Level Modeling and Instantiation

The 1-Bit Full Adder:

A Full Adder is a fundamental combinational logic block that performs the addition of three bits.



Task: Write the Verilog code for the 1-bit Full Adder using three different modeling views. You must **create three separate modules** and **only one testbench for all**:

1. **Gate-Level View:** Use built-in Verilog primitives (and, or, xor).
2. **Structural View:** Instantiate two Half Adders and an OR gate to construct the Full Adder. You will need to write the Half Adder module first.
3. **Behavioral View:** Use an always block or a continuous assignment (assign) using the concatenation operator for the addition.

NOTE:**HOW TO WRITE A TESTBENCH:**

- <https://vlsiverify.com/verilog/writing-a-testbench-in-verilog/>
- <https://chipverify.com/verilog/verilog-testbench>

Deliverables

Submit your **report** containing **all the answers** and your **.v files** containing all requested modules, alongside a **brief testbench (tb.v)** simulating the modules to verify correct operation.